

CSCE 614 COMPUTER ARCHITECTURE · MODULE 02 · LECTURE NOTES

Instruction Sets and the Hardware/Software Interface

The contract between the compiler and the chip.

MODULE OUTCOMES

- Explain what an ISA specifies and what it deliberately leaves open.
- Compare RISC and CISC design philosophies and their modern convergence.
- Read compiler-generated assembly well enough to diagnose performance.
- Explain addressing modes and the cost of each.
- Explain why the ISA is an abstraction boundary, and why that matters.

1 The instruction set as a contract

An instruction set architecture is the interface between software and hardware: the complete specification of what a program can observe about the machine it runs on. Its most important property is what it deliberately omits.

The ISA specifies	The ISA says nothing about
The instruction repertoire and the semantics of each instruction	How many cycles any instruction takes
Architectural registers: count, width, naming	How many physical registers exist, or register renaming
Addressing modes and memory semantics	Cache sizes, associativity, or whether caches exist at all
Data types and their bit representations	Pipeline depth, issue width, execution order
Exception, interrupt, and privilege behaviour	Branch prediction, speculation, prefetching

THIS SEPARATION IS DOING SERIOUS ECONOMIC WORK

Software compiled in 1985 still runs on a processor built in 2026, even though the two share essentially no implementation technique. The 2026 chip breaks those instructions into internal micro-operations, reorders them aggressively, executes them speculatively across a dozen functional units, and retires them in order so the program cannot tell.

Only the ISA was held fixed. Everything underneath changed completely, repeatedly, and invisibly.

Like every abstraction boundary, it leaks — and it leaks precisely where performance is concerned, which is why a course on what lies beneath it is necessary.

2 RISC and CISC

	CISC (x86, VAX, 68000)	RISC (ARM, RISC-V, MIPS, SPARC)
Instruction count	Hundreds, many specialised	Tens to low hundreds, each simple
Encoding	Variable length (1–15 bytes on x86)	Fixed width (typically 4 bytes)
Memory operands	Most instructions may access memory	Load/store only — arithmetic works on registers
Architectural registers	Few (8 in 32-bit x86, 16 in x86-64)	Many (32 is typical)
Original rationale	Memory was expensive, so dense code mattered; compilers were weak, so complex instructions were written by hand	Simple instructions pipeline cleanly and decode trivially; let the compiler build complexity

The RISC argument, made in the early 1980s, was that most of a complex instruction set goes unused, that variable-length decoding is a bottleneck, and that simple uniform instructions allow deeper pipelines and faster clocks. The argument was substantially correct.

HOW IT ACTUALLY RESOLVED

Modern x86 processors decode their complex variable-length instructions into simple fixed-format internal **micro-operations**, and execute those on what is recognisably a RISC core with register renaming and out-of-order issue. The CISC instruction set persists as a compatibility interface, not as an implementation strategy.

Meanwhile, RISC architectures have acquired vector extensions, cryptographic instructions, and other complex operations wherever they earn their silicon.

RISC won the engineering argument; CISC won the market through compatibility. The contradiction is only apparent: once you see the ISA as an interface rather than a blueprint, both outcomes follow naturally. The decode cost x86 pays is real but has become a small fraction of a large core's power and area.

3 Reading assembly

You will not write assembly. You will read it, because it is the only way to find out what the compiler actually did — and the compiler does a great deal that is invisible in the source.

```
// Source
int sum(int* a, int n) {
    int s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return s;
}

// x86-64, -O3, abbreviated
.vector_loop:
    vpaddd    ymm0, ymm0, [rdi+rax*4]      ; 8 ints per instruction
    vpaddd    ymm1, ymm1, [rdi+rax*4+32]   ; second accumulator breaks
    add       rax, 16                      ; the dependency chain
    cmp       rax, rdx
    jb        .vector_loop
```

From five lines of C the compiler produced a vectorised loop processing eight integers per instruction, unrolled by two, using two independent accumulators so that successive additions do not wait on each other, followed by a horizontal reduction and a scalar tail for the remaining elements. None of this is visible in the source, and all of it affects performance by large factors.

3.1 What to look for

Question	What to look for	If absent
Did it vectorise?	ymm/zmm registers, v-prefixed instructions	Something blocked it: aliasing, a branch, a reduction the compiler could not prove reassociable (Module 09).
Did it unroll?	The loop body repeated several times per branch	Loop overhead may dominate a short body.
Register pressure?	mov to and from [rsp+...] inside the loop	Spilling to stack. Reduce live variables.
Branches in the loop?	j* instructions in the body	Possibly convertible to branchless (Module 04).
Division present?	div, idiv	20–40 cycles. Often replaceable by multiplication or a shift.
Calls not inlined?	call in the hot loop	The compiler lacked visibility. Consider making the definition available.

Compiler Explorer (godbolt.org) shows source and generated assembly side by side with colour-coded correspondence, for any compiler and flags. It is the single most useful tool in this course and should become habitual.

4 Addressing modes and the real cost of memory

Mode	Example	Cost
Register	add rax, rbx	Free.
Immediate	add rax, 42	Free.
Base + displacement	mov rax, [rbp-8]	One memory access; the arithmetic is free.
Base + index×scale + disp	mov rax, [rdi+rsi*8+16]	Identical cost. A dedicated address generation unit computes this without consuming an arithmetic slot — array indexing is free.
Dependent indirection	mov rax, [rax]	Same nominal cost, but serialised : the next address is not known until this load returns.

POINTER CHASING: THE MECHANISM BEHIND A 50× DIFFERENCE

Address arithmetic costs nothing. What costs is a load whose address depends on a previous load's result. Scanning an array issues many *independent* loads. The memory system can have ten or more outstanding simultaneously, and the prefetcher can predict the pattern and fetch ahead, so the latencies overlap almost completely.

Traversing a linked list issues one load, waits up to ~250 cycles for it, and only then knows where to look next. Nothing overlaps and nothing can be prefetched, because the address is unknowable in advance.

This is the mechanism behind the 10–50× gap between arrays and linked lists at identical asymptotic complexity — the gap CSCE 629 Module 01 could only assert. It is not that each access is more expensive; it is that none of them overlap.

5 The abstraction leaks

The ISA tells you what an instruction computes, never how long it takes. Two functionally identical programs can differ in running time by a factor of fifty while both being entirely correct at the ISA level.

All of that difference lives in the part the ISA deliberately leaves unspecified: caches, pipelines, predictors, execution order. This is precisely why you cannot reason about performance from the instruction set, and why the remaining eleven modules are necessary.

SPECTRE: THE ABSTRACTION LEAKING IN A NEW DIRECTION

The Spectre and Meltdown vulnerabilities of 2018 exploited the fact that speculative execution leaves measurable traces in the cache even when the speculated instructions are architecturally discarded.

The ISA guarantees that mis-speculated instructions have no *architectural* effect, and that guarantee held. It says nothing about microarchitectural state, and that is where the information escaped.

The abstraction boundary had always leaked timing information. It turned out to leak confidentiality too — a reminder that an abstraction which is merely approximately true is a security property only by accident. Module 04 returns to this.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — ISA, RISC vs CISC lectures	The design-space discussion done properly, including why the historical debate resolved as it did.
MIT 6.004 — Instruction Sets, Assembly Language	Builds an ISA from scratch and then a processor to run it. The best way to understand what an ISA is for.
Compiler Explorer	Source and assembly side by side for any compiler and flags. Use it in every remaining module.
Agner Fog — Instruction tables	Latency and throughput for every instruction on every recent x86 microarchitecture. The reference when you need an exact number.
RISC-V specifications (free)	A modern ISA designed in the open, and short enough to read. Instructive by contrast with x86.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Compile a simple loop at -O0, -O1, -O2, and -O3 on Compiler Explorer. Describe what changes at each level and measure the running time of each.
- 2 Write a reduction loop and determine whether it vectorises. If it does not, find out why — floating-point reassociation and pointer aliasing are the usual culprits — and fix it with `restrict` or `-ffast-math`, noting the semantic cost.
- 3 Measure array traversal against linked-list traversal over the same number of elements, with the list nodes allocated in order and then shuffled. Explain all three results.

- 4 Write a loop with enough live variables to force register spilling. Find the threshold on your machine and observe it in the assembly.
- 5 Replace an integer division by a constant with a multiply-and-shift. Verify your compiler already does this, and measure the case where it cannot (a runtime divisor).
- 6 Compile the same C function for x86-64 and for ARM64 (AArch64) on Compiler Explorer. Compare instruction counts and comment on the RISC/CISC differences you can actually observe.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name four things an ISA specifies and four it deliberately does not.
- 2 Why can a 1985 binary run on a 2026 processor that shares none of its implementation?
- 3 How did the RISC/CISC debate resolve, and why is 'both won' not a contradiction?
- 4 Give four things to look for when reading compiler output for performance.
- 5 Is $[rdi + rsi*8 + 16]$ more expensive than $[rdi]$? Explain.
- 6 Explain precisely why pointer chasing is slow, in terms of dependence rather than cost per access.
- 7 In what sense is the ISA a leaky abstraction, and what did Spectre demonstrate about that?